

Technical Note

CTXP Format

Version 1.0.0 ~ Build ea732979
2026-01-12



ACCEMIC
TECHNOLOGIES

Table of Contents

Revision History	1
1. Licensing and Usage	2
2. Disclaimer	3
3. C-Trace eXPort (CTXP) - File Format Specification (PR8.18)	4
3.1. Introduction	4
3.2. Data Model & Semantics	5
3.3. Trace Events	7
3.4. Text Encoding (.ctxp.txt)	9
3.5. Binary Encoding (.ctxp)	10
3.6. Example	12
3.7. Cross-source Synchronization	13
3.8. Limitations / Future Ideas	13
3.9. Practical Tips for Tool Developers	14
3.10. Export from Native Traces	14
4. Acknowledgment	15

Revision History

Version	Date	Changes
1.0.0	2026-01-12	Initial release.

Chapter 1. Licensing and Usage

This document is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

You are free to share (copy and redistribute the material in any medium or format) and adapt (remix, transform, and build upon the material) for any purpose, including commercial use, provided that appropriate credit is given, a link to the license is provided, and any changes made are indicated.

Usage of this document is subject to the following conditions:

- Attribution is required. Any use or distribution of this document, or derivatives thereof, must include clear attribution to the original authors and source.
- Commercial use is allowed, but the authorship and source must always be referenced in any publication or product that incorporates this material.
- No additional restrictions. You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

The full license text is available at: <https://creativecommons.org/licenses/by/4.0/>

Chapter 2. Disclaimer

This document is provided “as is”. The authors and publishers make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability of this document or the information contained herein. Any reliance you place on such information is strictly at your own risk. The authors and publishers do not accept any liability for any loss or damage, including without limitation, indirect or consequential loss or damage, arising from the use of this document.

Chapter 3. C-Trace eXPort (CTXP)

- File Format Specification (PR8.18)

3.1. Introduction

The CXP file format defines a unified format for processor trace data: control-flow events, memory accesses, and timing information from a single multi-core system. This document describes file version 1.

In principle, the format lifts native bit-compressed hardware traces like RISC-V C-Trace, ARM CoreSight ETMv4 or TriCore MCDS, etc. into a uniform byte-level event stream, that is easier to digest for trace analyzers. Although similar vendor-specific formats exist, no public available format has yet been defined, unlike the CXP format.

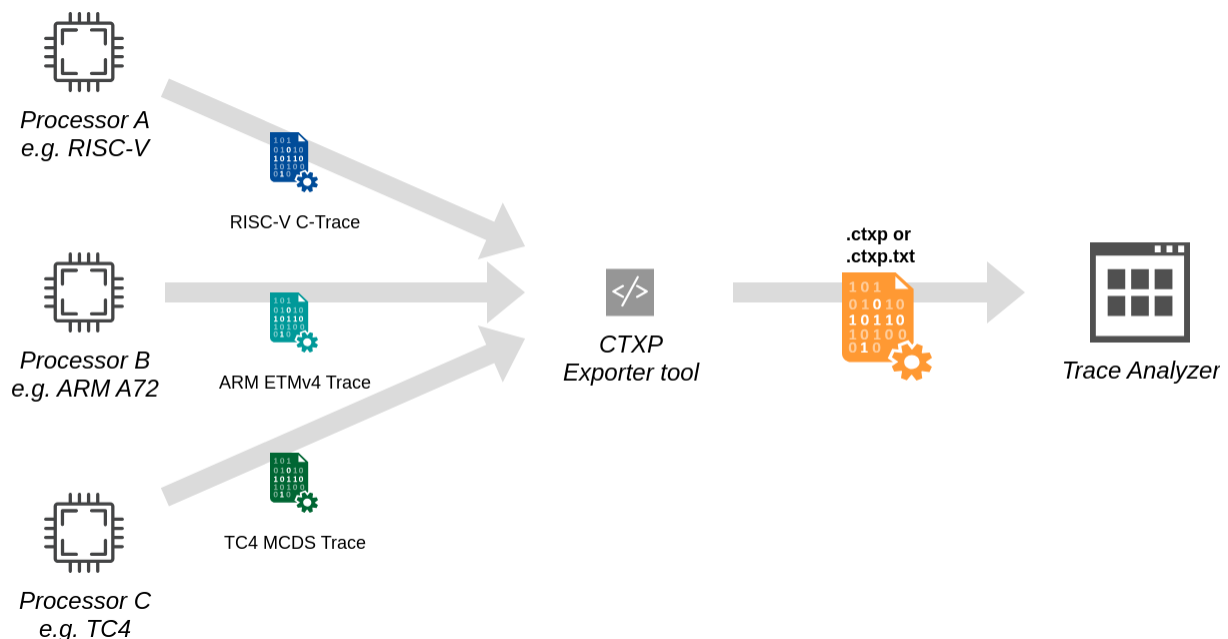


Figure 1. CXP format provides a unified input for trace analyzers.

Key Features:

- Multiple trace sources (up to 256)
- Represent control-flow events (branches, calls, returns, interrupts, return-from-interrupt)
- Control-flow has 2 modes: function trace and branch trace
- Represents memory accesses (reads/writes of 0 (address only), 1, 2, 4, or 8 bytes with address & value)
- Trace-local per-core cycle timestamp for precise timing (optional)

- Optional global timestamp markers for cross-source correlation
- Dual representations: binary (.ctxp) and text (.ctxp.txt)
- Easily processable, grep-able text format, understandable by humans

Typical use cases for trace analyzers include post-mortem debugging, performance profiling, and visual analysis. These tasks involve various processing steps such as converting CTXP addresses into symbols and source code locations, performing cross-source synchronization, or even aligning trace data with other system events.

Important Remarks:

- The set of recorded events depends entirely on the trace configuration of the target system.
- When speaking of "instruction address", it means the first byte associated with the instruction.

3.2. Data Model & Semantics

The data model is equal for text and binary encoding.

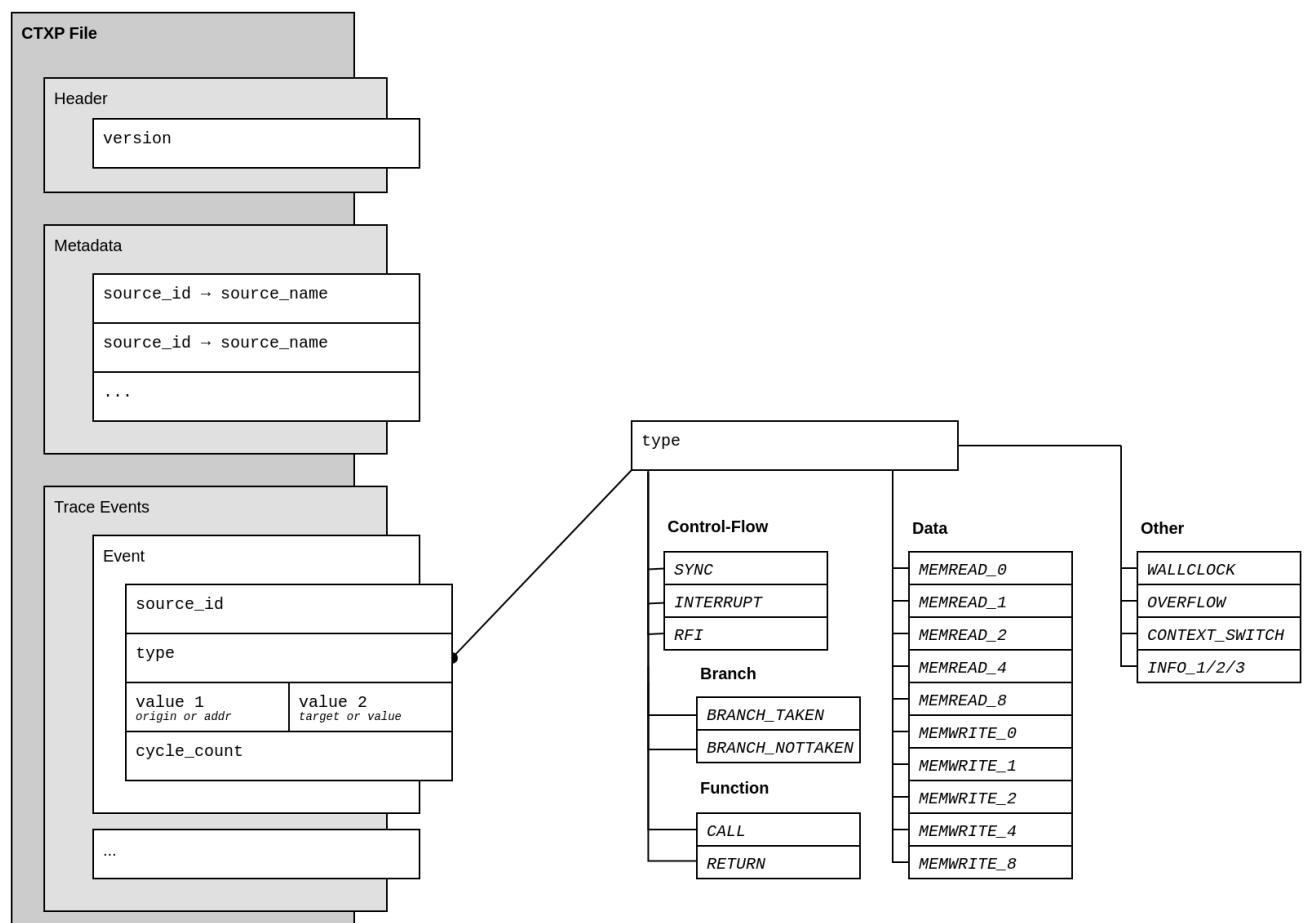


Table 1. Each field in the data model has a defined type.

Field	Type	Value	Description
version	16 bit unsigned	0x1	1 = Initial Version
source_id	8 bit unsigned		Numeric identifier of which trace source emitted the event.
source_name	UTF8 string		Text description for <code>source_id</code> . Maximum 1024 bytes.
type	8 bit enum		Event type. Defines semantics of <code>value 1</code> and <code>value 2</code> .
value 1/2	64 bit		Event payload. See details in section Section 3.3 about exact semantics per <code>type</code> .
cycle_count	64 bit unsigned		Absolute source-local, monotonic cycle count. Optional. Does not wrap. <code>OVERFLOW</code> events resets the count.

Table 2. The event type defines the validity of the two payload fields, and their basic semantics.

Type	Payload Value 1	Payload Value 2
SYNC		target
BRANCH_TAKEN	origin	target
BRANCH_NOTTAKEN	origin	target
CALL	origin	target
RETURN	origin	target
INTERRUPT	origin	target
RFI	origin	target
MEMWRITE_0	addr	
MEMWRITE_1	addr	value
MEMWRITE_2	addr	value
MEMWRITE_4	addr	value
MEMWRITE_8	addr	value
MEMREAD_0	addr	
MEMREAD_1	addr	value
MEMREAD_2	addr	value
MEMREAD_4	addr	value
MEMREAD_8	addr	value
OVERFLOW		
CONTEXT		value
WALLCLOCK		value

3.2.1. Metadata

- Map of human-readable names for decimal-encoded source ids, in the form
 - Source Id → Source name (byte to string)
- A source name must not be given for every source id.

- A source id should not appear twice in the map.

3.3. Trace Events

Event Ordering

Events are ordered chronologically per source by `cycle_count`.

When multiple sources are interleaved, their relative order is undefined unless `WALLCLOCK` synchronization points are present. If no `cycle_count` is given, the order still follows the order of their appearance in the trace-event section. Anyhow, events might have happened simultaneously.

3.3.1. Control-Flow Events

Control-flow events allow to trace back, which code a CPU core executed. In principle, the instruction trace is **continuous**, meaning that intra-event points can be inferred from looking at two events. Only `OVERFLOW` events create a discontinuity. In the special case of function-only traces, fine-grained branch information cannot be inferred by the subsequent trace analyzer.

SYNC

Core with `source_id` reached instruction at `target` address. Typically the first control-flow related event setting the starting point of the trace. Might also deliver additional support points for timing information. For example - as `BRANCH_TAKEN` events only carry timing about when `target` got reached, an additional `SYNC` could give the time the actual branch instruction was reached. The optional `cycle_count` specifies when `target` got reached.

INTERRUPT

Core with `source_id` reached `target` address because of an interrupt. The last known instruction address that got executed, or at least tried to be executed (for example for invalid-opcode) before the interrupt is given by `origin`, and the next known address `target` is the instruction where the core continues. The optional `cycle_count` specifies when `target` got reached.

RFI

Short for return-from-interrupt. CPU came back from an interrupt. The of last known executed instruction address is given at `origin`, and the first known instruction address where the CPU continues is `target`.

BRANCH_TAKEN

Core with `source_id` reached `target` address after taking a branch instruction. All types of branches are considered - may they be conditional, unconditional, direct or indirect, **except for calls & returns** (see `CALL/RETURN`). `origin` is the address of the branch instruction, while `target` is the destination of the branch instruction. The optional `cycle_count` specifies when `target` got reached.

BRANCH_NOTTAKEN

Core with `source_id` reached `target` address after skipping a conditional branch (may it be direct or indirect). `origin` is the address of the branch instruction, and `target` the address of the reached instruction directly after the skipped branch. The optional `cycle_count` specifies when `target` got reached.

CALL

Core with `source_id` reached `target` address by a call instruction. `origin` is the callers instruction address. The optional `cycle_count` specifies when `target` got reached.

RETURN

Core with `source_id` reached `target` after executing a return instruction. `origin` represents the instruction address of the return. The optional `cycle_count` specifies when `target` got reached.

3.3.2. Data Events

MEMREAD_0/MEMWRITE_0

Source with `source_id` finished a read/write access to address `addr` of unknown size and data. No actual data is logged. The optional `cycle_count` specifies when the transaction finished.

MEMREAD_1,2,4,8

Source with `source_id` performed a read access at `address`, with `value`. Could be 1, 2, 4 or 8 bytes size. The endianness of `value` fields follows the endianness of the traced target. The optional `cycle_count` specifies when the transaction finished.

MEMWRITE_1,2,4,8

Source with `source_id` performed a write access to `address`, with `value`. Could be 1, 2, 4 or 8 bytes size. The endianness of `value` fields follows the endianness of the traced target. The optional `cycle_count` specifies when the transaction finished.

3.3.3. Other Events - Overflows, Timestamps, etc.

OVERFLOW

Signals the source-local trace buffer wrapped or lost events for `source_id`. Source's `cycle_count` of following events will restart from zero. Decoder must totally reset its current state of the affected source.

CONTEXT

Indicates a new context or task of the CPU core with `source_id`. New context identifier is given in `value`. The semantics of the identifier is defined by the native trace protocol, and is typically a task id. The subsequent decoder might use this event to analyze the current tasks. For processors with virtual address space, a new context might also result in a change of the

related program binary.

WALLCLOCK

Global, cross-source available timestamp update to `value`. The optional `cycle_count` sets the relation to the source-local time.

INFO_1,2,3

Unspecified, user-defined events. Allow flexibility by exporter and analyzer, and shall be gently ignored by all parsers. May carry optional `cycle_count` indicating when the event occurred.

3.4. Text Encoding (.ctxp.txt)

The text representation of CTXP-files is mainly for human interaction, but is also machine-readable. CTXP text files must use UTF8-no-BOM encoding. Line breaks must be LF.

- First line must be the header with version:

```
1 HDR:format=accemic//ctxp-txt,ver=1
```

- The metadata section must follow the following example format. Source names must be quoted. Source names containing a quote-symbol " must escape the symbol with a backslash \ like \ ". Backslashes must be escaped with another preceding backslash like \\. Source ids are decimal.

```
1 META:#0="CPU0",#1="CPU1",#2="CPU2",#3="CPU\\\\"3\""
```

- Followed by the trace data, as list of events in each line:

```
1 #<SOURCE ID:dec>:<TYPE:string>:(<VALUE 1:hex>)?:(<VALUE 2:hex>)?:(@ <CYCLE COUNT:dec>)?
```

- `TYPE:string` must be a type defined in [Section 3.3](#).
- Presence of `VALUE 1` and `VALUE 2` depend on `TYPE`, defined in [Table 2](#). Colons are always present.
- `CYCLE_COUNT` is optional, and indicated by `@` at the last part of the line.
- To improve readability, whitespaces (space and tab) are allowed between the elements, so parsers so gently skip them.
- Hex-values must have `0x` prefix, are lower-case, and can contain preceding zeros, so they have a variable length.

.ctxp.txt example with explanation:

```
1 HDR:format=accemic//ctxp-txt,ver=1
2 META:#0="CPU0",#1="CPU1"
```

3	#0:SYNC::0x80000000	@	0	①
4	#0:BRANCH_NOTTAKEN:0x80003ca2:0x80003ca6	@	4	②
5	#1:MEMREAD_1:0x70000064:0x00	@	1	③
6	#0:WALLCLOCK::0x12000	@	10	④
7	#1:WALLCLOCK::0x12000	@	4	⑤
8	#0:BRANCH_TAKEN:0x80003d00:0x8000298c	@	24	⑥

- ① Core 0 currently at PC 0x80000000, cycle=0
- ② Core 0, non-taken branch at 0x80003ca2 was skipped, and continue with 0x80003CA6, cycle=4
- ③ Another core, CPU1, performed a 1-byte read at addr=0x70000064, value=0x0, cycle=1
- ④ Global timestamp is now 73728 (0x12000), at CPU0 cycles=10
- ⑤ Global timestamp is now 73728 (0x12000), at CPU1 cycles=4
- ⑥ Core 0, taken branch at 0x80003d00 was taken to PC 0x8000298C, cycles=24

3.5. Binary Encoding (.ctxp)

The binary encoding aims to provide a fast machine-consumption of the trace.

- All multi-byte values are little endian.

3.5.1. Header Encoding

The .ctxp file starts with a fixed-size header:

Table 3. Header Section

Field	Size (bytes)	Description
Magic	4	ASCII string "CTXP" (0x43 0x54 0x58 0x50)
HeaderSize	2	Length of this header in bytes (for v1: 8)
Version	2	File format version (for v1: 1)

Example .ctxp Header (hex dump)

43 54 58 50 08 00 01 00
CTXP 8 v1

3.5.2. Metadata Encoding

Field	Size (bytes)	Description
SectionType	2 (LE)	Section identifier (0x0001 for metadata)
SectionLength	2 (LE)	Total section length in bytes, including starting from SectionType
Payload	value(SectionLength)	List of entries
> SourceId	1	Numeric source identifier
> NameLen	1	Length of name string in bytes (UTF-8)
> Name	NameLen	UTF-8 encoded string (without terminator)

Example (full metadata section with one entry)

```
01 00 ; SectionType = 1 (metadata)
11 00 ; SectionLength = 17 bytes
01 09 63 6F 72 65 5F 63 70 75 30 ; id=1, "core_cpu0"
```

3.5.3. Event Encoding

- TraceEvents are packed to exactly 26 bytes each. Events are tightly packed, with no padding between entries.
- Multi-byte values are in general little-endian, except for `data.value`, which endianness depends on target architecture.

```
1 typedef struct attribute((packed)) {
2     uint8_t      source_id;
3     EventType    type;          // See enum below.
4     union {
5         struct { uint64_t origin, target; } instr;
6         struct { uint64_t addr, value; } data;
7     } payload;
8     uint64_t      cycle_count;  // Optional. Valid if MSB of type is set.
9 } TraceEvent;
10
11 enum class EventType : uint8_t
12 {
13     // CPU control flow
14     SYNC = 0b0'000'0000,
15     SYNC_WITH_TIMESTAMP = 0b1'000'0000,
16     BRANCH_TAKEN = 0b0'001'0001,
17     BRANCH_TAKEN_WITH_TIMESTAMP = 0b1'001'0001,
18     BRANCH_NOTTAKEN = 0b0'001'0010,
19     BRANCH_NOTTAKEN_WITH_TIMESTAMP = 0b1'001'0010,
20     INTERRUPT = 0b0'001'0011,
21     INTERRUPT_WITH_TIMESTAMP = 0b1'001'0011,
22     RFI = 0b0'001'0101,
23     RFI_WITH_TIMESTAMP = 0b1'001'0101,
24     CALL = 0b0'001'0110,
25     CALL_WITH_TIMESTAMP = 0b1'001'0110,
26     RETURN = 0b0'001'0111,
27     RETURN_WITH_TIMESTAMP = 0b1'001'0111,
28
29     // Memory write events
30     MEMWRITE_0 = 0b0'010'0000,
31     MEMWRITE_0_WITH_TIMESTAMP = 0b1'010'0000,
32     MEMWRITE_1 = 0b0'010'0001,
33     MEMWRITE_1_WITH_TIMESTAMP = 0b1'010'0001,
34     MEMWRITE_2 = 0b0'010'0010,
35     MEMWRITE_2_WITH_TIMESTAMP = 0b1'010'0010,
36     MEMWRITE_4 = 0b0'010'0100,
37     MEMWRITE_4_WITH_TIMESTAMP = 0b1'010'0100,
38     MEMWRITE_8 = 0b0'010'1000,
39     MEMWRITE_8_WITH_TIMESTAMP = 0b1'010'1000,
40
41     // Memory read events
42     MEMREAD_0 = 0b0'011'0000,
43     MEMREAD_0_WITH_TIMESTAMP = 0b1'011'0000,
44     MEMREAD_1 = 0b0'011'0001,
45     MEMREAD_1_WITH_TIMESTAMP = 0b1'011'0001,
46     MEMREAD_2 = 0b0'011'0010,
47     MEMREAD_2_WITH_TIMESTAMP = 0b1'011'0010,
48     MEMREAD_4 = 0b0'011'0100,
49     MEMREAD_4_WITH_TIMESTAMP = 0b1'011'0100,
```

```

50 MEMREAD_8 = 0b0'011'1000,
51 MEMREAD_8_WITH_TIMESTAMP = 0b1'011'1000,
52
53 // Trace control
54 OVERFLOW = 0b0'101'1111,
55
56 // Others
57 CONTEXT = 0b0'100'0000,
58 CONTEXT_WITH_TIMESTAMP = 0b1'100'0000,
59 WALLCLOCK = 0b0'100'0001,
60 WALLCLOCK_WITH_TIMESTAMP = 0b1'100'0001,
61
62 // User-defined
63 INFO1 = 0b0'111'0000,
64 INFO1_WITH_TIMESTAMP = 0b1'111'0000,
65 INFO2 = 0b0'111'0001,
66 INFO2_WITH_TIMESTAMP = 0b1'111'0001,
67 INFO3 = 0b0'111'0010,
68 INFO3_WITH_TIMESTAMP = 0b1'111'0010
69 };

```

3.6. Example

Disassembly of section .text:

80000000 <_start>:	#HDR:format=accemic//ctxp-txt,ver=1
80000000: auipc t3,0x0	#META:#0="core0"
80000004: add t3,t3,68 # 80000044	#0:SYNC: :0x80000000 @ 0
80000008: csrw mtvec,t3	#0:BRANCH_NOTTAKEN:0x80000014:0x80000018
8000000c: li t0,1	#0:CALL:0x8000001c:0x8000004c @ 15
80000010: li t1,1	#0:RETURN:0x80000050:0x80000020 @ 18
80000014: beq t0,t1,8000001c <_start+0x1c>	#0:SYNC: :0x80000024 @ 19
80000018: nop	
8000001c: jal 8000004c <foo>	
80000020: auipc t2,0x0	
80000024: add t2,t2,12 # 8000002c	
80000028: jr t2	
...	
8000004c <foo>:	
8000004c: li a0,42	
80000050: ret	

Figure 2. The CTXP trace allows reconstructing the path taken through the example program by a single CPU core.

Explanation:

- Program reached 0x8000_0000 at time 0.
- Program took 15 clock cycles to reach 0x8000_004c by a call, executing all instructions since, and including 0x8000_0000, and including the nottaken branch at 0x8000_0014, but without actually executing the li a0, 42 instruction at 0x8000_004c.
- Program took 3 cycles to return to 0x8000_0020.
- Program took 1 cycle to execute the auipc t2, 0x0 instruction at 0x8000_0020, reaching 0x8000_0024.

3.7. Cross-source Synchronization

Cross-source synchronization is the process of bringing events from two or more sources to a global order, of what happened when.

The `WALLCLOCK` events serve as anchor for all sources. The associated `cycle_clock` defines the offset to each source timing.

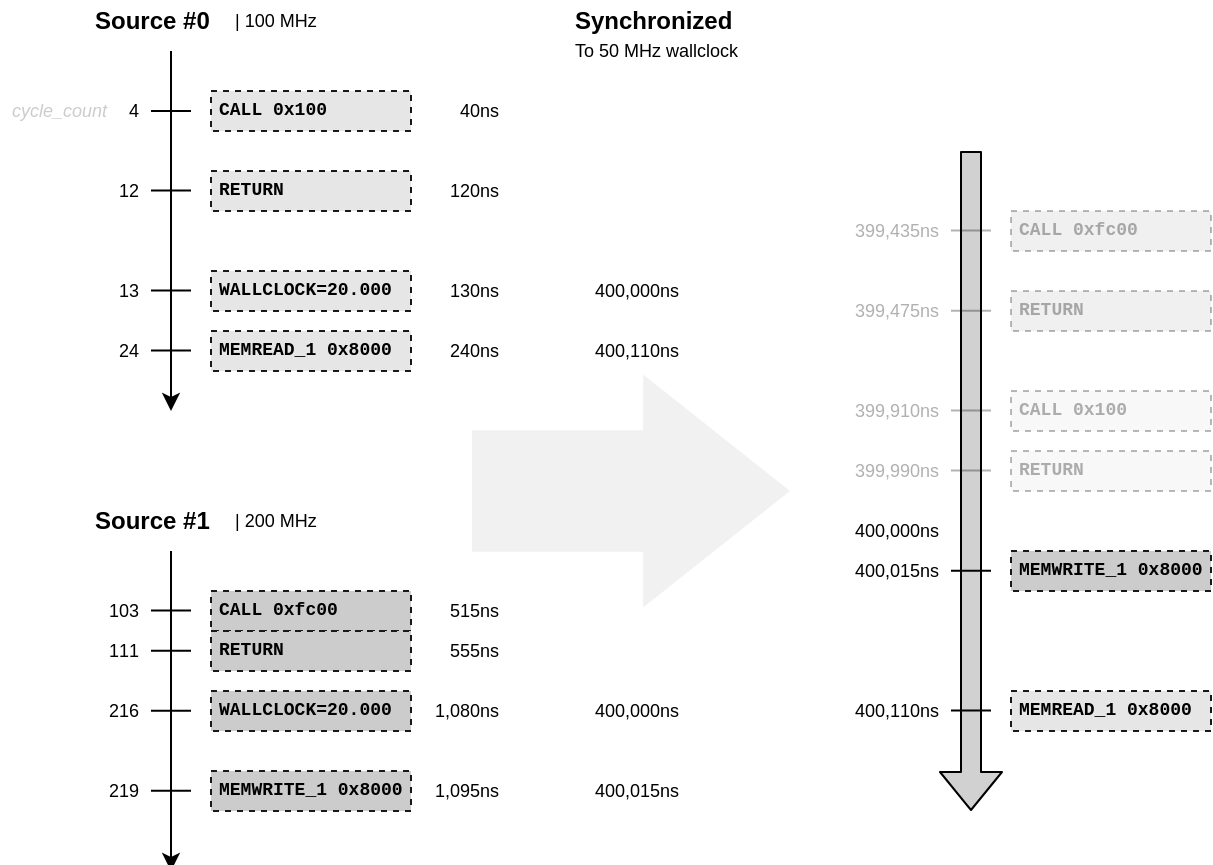


Figure 3. Cross-source synchronization allows to determine which events happened before others. Often, sources and wall clock run on different frequencies.

The following steps are required for the synchronization:

1. Convert all source `cycle_count` and `WALLCLOCK` values to a common base, like nanoseconds.
2. For each source, apply the `WALLCLOCK`'s `cycle_count` offset to all events.
3. Order all events from all sources on the global time value.

3.8. Limitations / Future Ideas

- No dynamic frequency/power-state events
- Declare observed instruction address regions to support filtered traces. Could be implemented by defining a new section type.

- Include cycle-count frequencies for conversion. Could be implemented later through a global header or individual events.
- Traces are theoretically limited in time by the 64-bit `cycle_count` to around 500 years length (CPU clock of 1 GHz).

3.9. Practical Tips for Tool Developers

- Prefer compressed .ctxp files (e.g. ZIP) to reduce disk size.
- Unknown format versions must be rejected, as they may contain breaking changes.

3.10. Export from Native Traces

Converting native embedded traces requires in-depth knowledge about the individual formats. In the following sections, some thoughts are given for translating various formats. [Accemic](#) offers a universal CTXP exporter tool [on request](#), which can decode all modern native traces.

3.10.1. RISC-V C-Trace

- Branch-history based traces encode multiple branches, and, hence, result in multiple CTXP trace events.

3.10.2. TriCore MCDS

- Branch-level granularity reduces timing insight for TriCore, usually it gives times like x ticks until indirect branch, and then 6 cycles to **perform** the branch. As we only operate on basic blocks, we lose that fine-grained information.
- If data accesses are traced, but not actual data, `MEMWRITE_0/MEMREAD_0` events are exported.
- Interrupts at TriCore can hardly be completely traced. Entry point might be delayed. DCU messages need to be traced.
- The CFT (compact function trace mode) results in bare `CALL/RETURN` events. No branch-related events would be exported.
- Task switches are typically identified by tracing dedicated memory address, as the TriCore architecture does not have a dedicated task/context id register. The CTXP exporter might convert those memory trace events to `CONTEXT` events.

3.10.3. ARM PFT / ETMv4

- Multi-atom packets multiple branches, and, hence, result in multiple CTXP trace events.

Chapter 4. Acknowledgment

The CTXP format was developed as part of the TRISTAN project, a European Union research initiative involving 46 partners to advance the RISC-V ecosystem. See <https://tristan-project.eu/> for more information.